Module 7: Building AI Systems

Inputs, generation, tools, evaluation, and artifacts

Module 7: Building AI Systems

Primary sources: Osmani, Chapter 7; Thomas & Hunt, Topic 12: Tracer Bullets.

Learning Outcomes

- Analyze components of an AI-assisted system.
- Describe data and context flow.
- Evaluate integration challenges.
- Design a basic architecture for an AI-assisted feature.

From Task to System

An AI-assisted system is more than a model call.

It includes:

- Input handling.
- Context assembly.
- Prompt construction.
- Model interaction.
- Tool invocation.
- Evaluation.
- Artifact tracking.
- Human oversight.

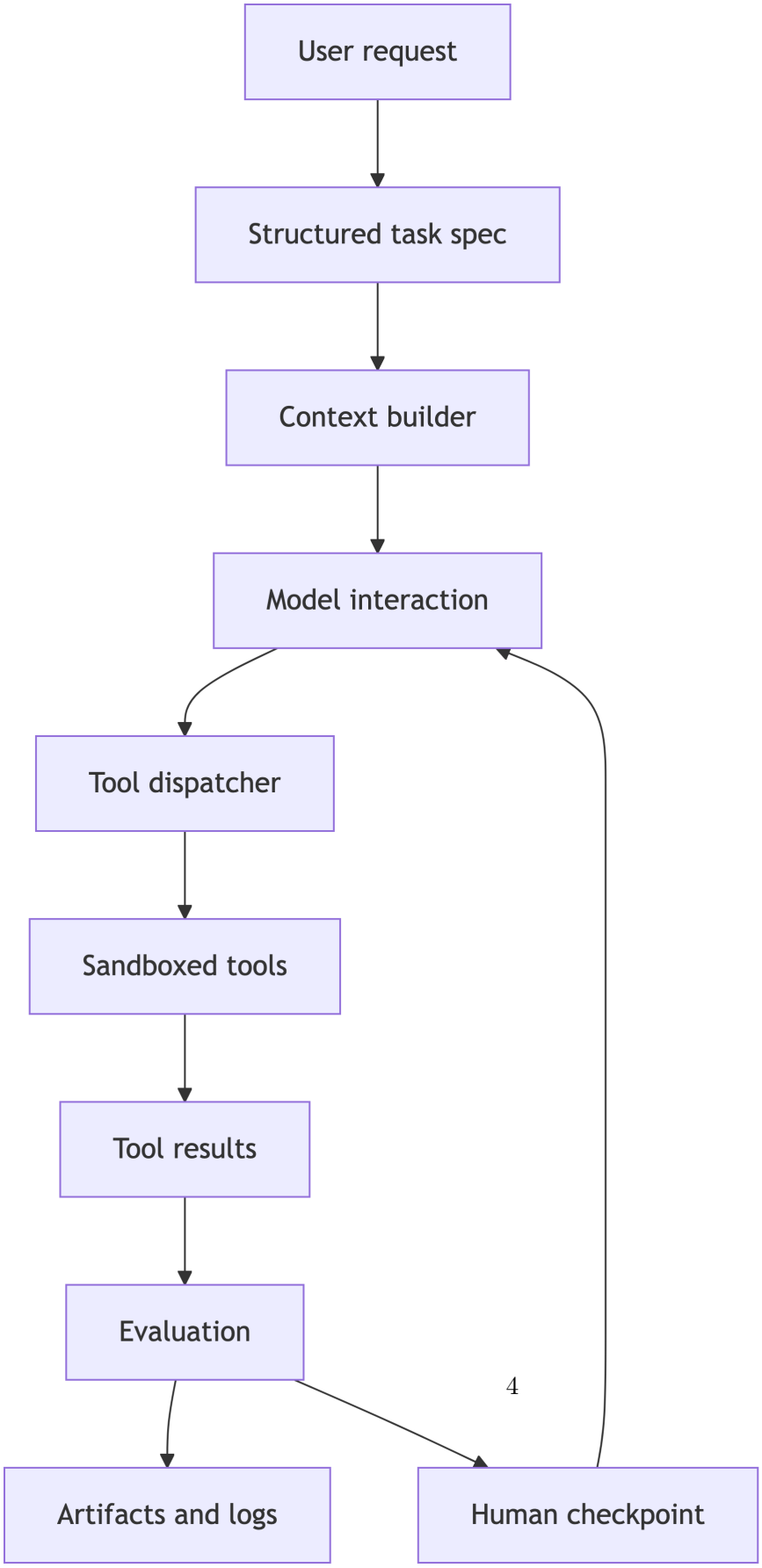
Full-Stack AI Workflow

Osmani Chapter 7 frames AI-assisted development as an end-to-end workflow. In this course, treat full-stack examples as a transfer pattern for your controlled agent.

- Scaffold the project and dependency structure.
- Generate frontend components from clear descriptions.
- Implement backend routes, business logic, and validation.
- Integrate data storage and database access.
- Align frontend and backend contracts.
- Test and validate the whole stack.

For the Java agent project, the analogous stack is CLI input, context builder, model call, tool dispatcher, sandboxed file tools, evaluator, logs, and human checkpoint.

Controlled Agent Architecture



System Boundary

Define what the agent may and may not do.

For a controlled coding agent:

- Allowed tools.
- Allowed paths.
- Allowed commands.
- Required approvals.
- Required logs.
- Required tests.

Context Flow

Context should be curated, not dumped.

Typical flow:

1. User request.
2. Structured task specification.
3. Relevant project files or summaries.
4. Tool definitions.
5. Evaluation criteria.
6. Output and artifact records.

Integration Challenges

- Mismatched interface contracts.
- Unclear task or artifact models.
- Generated code that bypasses abstractions.
- Tests that require unavailable services.
- Tool outputs too verbose to inspect.
- Error handling split across layers.

Contracts Between Layers

AI can generate each layer quickly, but humans must keep the interfaces coherent.

Check:

- Command names, arguments, and result codes.
- Request and response schemas for model and tool calls.
- Artifact fields, log formats, and validation rules.
- Authorization assumptions around files and commands.
- Error formats consumed by the next workflow step.
- Test data that represents real agent states.

Data and Context Flow

In an AI-assisted system, context moves through several forms.

- User intent becomes task specification.
- Task specification becomes model prompt and tool constraints.
- Tool constraints become dispatcher behavior and sandbox rules.
- Runtime behavior becomes logs, tests, and evaluation artifacts.
- Evaluation results become the next prompt or code change.

Breakage usually appears at the boundaries, not inside the generated snippet.

Tracer Bullet Architecture

Build one thin complete workflow first.

Example:

- Read task.
- Ask model for plan.
- Dispatch one safe tool.
- Capture result.
- Evaluate with one check.
- Save run artifact.

The goal is not complete functionality; it is a real path through the system that exposes integration risk.

Testing the Whole Workflow

Osmani emphasizes validation across the stack, not just isolated code.

- Unit-test business logic and parsing.
- Integration-test API and database behavior.
- End-to-end-test the user-visible workflow.
- Verify generated queries against expected data shapes.
- Keep AI-created tests under the same review standard as code.

AI System Prompting

Useful prompts ask the model to operate inside architecture.

- "Use only these tools."
- "Return JSON matching this schema."
- "If input is ambiguous, ask for clarification."
- "Do not modify files outside this sandbox."
- "Explain verification steps separately from implementation."

Lab 4 Development Connection

Lab 4 begins risk analysis and production readiness.

Use a compact threat model before you implement more automation:

- Asset: what must be protected?
- Entry point: where can bad input enter?
- Trust boundary: where does control change hands?
- Mitigation: what prevents or contains failure?

Your risk analysis should be specific to your actual system:

- Tool misuse.
- Sandbox escape.
- Prompt injection.
- False confidence from weak tests.
- Missing human checkpoint.

Practice Activity

Draw your agent as boxes and arrows.

Submit the diagram with a short table. For each arrow, label:

- Data passed.
- Trust boundary.
- Validation step.
- Failure mode.

Takeaways

- AI engineering is systems engineering.
- Context, tools, and evaluation are first-class components.
- Thin end-to-end slices make risk visible.